Generating SQL from Natural Language with a Fine-Tuned Transformer

Project 5: Generative AI Applications — Generative AI Analysis and Ethics Report

Ewald Geschwinde

Overview

This project builds a conditional text generator that translates an English question about a database into an executable SQL query. The input is the question together with the table schema, written as a *CREATE TABLE* statement; the output is a SQL *SELECT* statement generated token by token. Generative modelling learns a probability distribution over data so that new samples can be drawn from it. For sequences, the dominant approach is the transformer, which factorises that distribution autoregressively and uses attention to relate every position to every other (Vaswani et al., 2017). We fine-tune CodeT5-small, a 60-million-parameter encoder–decoder pretrained on source code (Wang et al., 2021), on 70,715 human-written question/SQL pairs. Generative adversarial networks (Goodfellow et al., 2014) and variational autoencoders (Kingma & Welling, 2014) were not chosen. GANs cannot easily back-propagate through discrete token choices, and text VAEs are prone to posterior collapse, where the decoder ignores the latent code (Bowman et al., 2016). Diffusion models (Ho et al., 2020) dominate image synthesis but are less mature for exact, structured outputs such as code.

Dataset or Prompt Description

The data come from *sql-create-context* (b-mc2, 2023), a publicly available Hugging Face dataset (CC BY 4.0) of 78,577 triples: question, schema and gold SQL. It merges two established human-annotated benchmarks. WikiSQL (Zhong et al., 2017) contributes about 94% of examples: crowdworker-written questions over single Wikipedia tables, answered by simple SELECT–WHERE queries with an optional aggregate. Spider (Yu et al., 2018) contributes about 6%: complex, cross-domain queries written by Yale students, of which 39% use JOIN, 26% GROUP BY and 15% nested SELECTs. The dataset's curator added the schema by deterministically parsing each gold query for the tables and columns it uses, with coarse types (INTEGER or VARCHAR). No part of the data is AI-generated. Inputs are short, with a median of 41 encoder tokens and 25 target tokens.

The dataset supports the task directly: every question is paired with exactly the schema needed to answer it, so the model must learn to ground natural language in identifiers and copy values. Three properties shape the results. First, many WikiSQL questions share a table, so we split by schema (90/5/5): every one of the 3,941 test examples uses a schema never seen in training, which prevents leakage. Second, the schemas list only the columns used, which is easier than real databases with many distractor columns. Third, the annotations are noisy: 2.6% of the test-set gold queries fail to execute against their own schema. Some of these are conversion artefacts; for example, "smaller than 3,284" became *< 3 OFFSET 284*.

Model Design and Training Approach

CodeT5-small follows the T5 encoder–decoder design (Raffel et al., 2020). It has six encoder and six decoder layers with a model width of 512, 8 attention heads, 2,048-unit feed-forward layers, RMS layer

normalisation and learned relative position biases. The decoder's cross-attention lets each generated token attend to the question and schema, which is how column names and literal values are copied. Three design decisions were deliberate. (1) **Transfer learning:** 70k examples are too few to learn English and SQL from scratch, and fine-tuning a pretrained text-to-text model is the established remedy (Raffel et al., 2020). (2) **A code-pretrained model and tokenizer:** CodeT5's identifier-aware pretraining targets exactly the copying of variable-like names that SQL requires (Wang et al., 2021), and its byte-level BPE vocabulary encodes every SQL symbol. The original T5 vocabulary has no '<' character, so it could never write *WHERE x < 5*. (3) **Encoder–decoder rather than decoder-only:** the schema is read bidirectionally before generation begins.

Training minimised token-level cross-entropy with teacher forcing, ignoring padding positions. We used AdamW with decoupled weight decay of 0.01 (Loshchilov & Hutter, 2019) and a peak learning rate of 3×10^{-4}, with 5% linear warm-up then linear decay, gradient clipping at 1.0, batch size 32 and 3 epochs (6,630 steps, 72 minutes on an Apple M1 Pro GPU). The checkpoint with the lowest validation loss was kept. Validation loss fell from 4.68 before training to 0.046, and validation exact match rose from 78% to 81% to 83% across the three epochs (Figure 1), with no divergence or overfitting. The main practical difficulty was memory: dynamically padded batches produced many distinct tensor shapes, and the Apple-GPU allocator cached memory for each of them until the process reached 34 GB and the machine began swapping. Padding to multiples of 16 tokens and periodically clearing the cache fixed this. Outputs were decoded with 4-beam search.

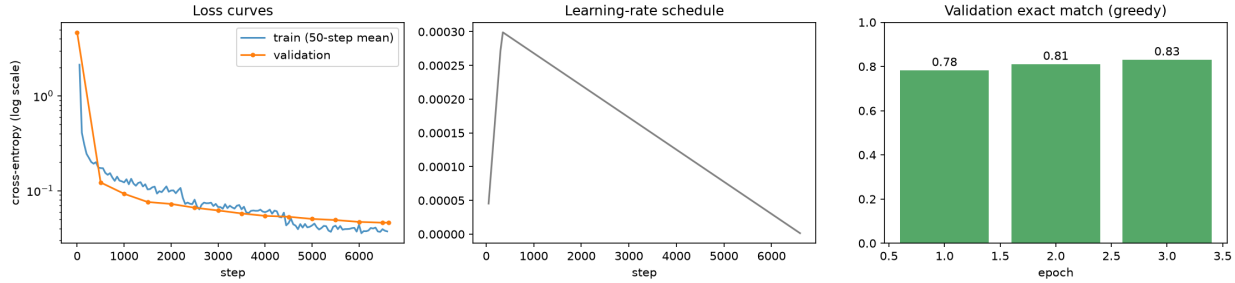


Figure 1. Training and validation cross-entropy (log scale), learning-rate schedule, and validation exact match after each epoch.

Output Evaluation and Interpretation

On the unseen-schema test set, 96.2% of generated queries are valid SQL, 82.0% match the reference exactly after normalisation, and 88.5% return the same result as the reference on a randomly populated SQLite database (Table 1). Execution matching follows the idea of test-suite evaluation (Zhong et al., 2020) but uses a single database, so it slightly overestimates semantic accuracy. Before fine-tuning, the model produced only fragments such as *'def'*, so all task competence was learned in fine-tuning. Quality is strong on single-table queries: the model copies unusual identifiers such as *table_27067379_1* and literal values such as *"7–5–3"* exactly. Quality degrades sharply as structure grows: Spider queries reach 50.8% exact match and JOIN queries only 16.0%.

Test subset	n	Exact match	Valid SQL	Execution acc.
All	3941	82.0%	96.2%	88.5%
WikiSQL	3699	84.0%	97.6%	90.1%
Spider	242	50.8%	73.6%	63.3%
aggregate	1171	69.4%	92.1%	79.1%
ORDER BY	61	45.9%	68.9%	56.9%
GROUP BY	65	32.3%	70.8%	45.2%
JOIN	119	16.0%	50.4%	32.7%

Table 1. Test-set performance by subset (3,941 examples on unseen schemas; beam search).

Concrete failure case. For *"Show the names of pilots and models of aircrafts they have flied with"*, every distinct sampled query was invalid SQL. Most used the correct three-table join skeleton but attached columns to the wrong table aliases. One example:

```
gold: SELECT T3.Pilot_name, T2.Model FROM pilot_record AS T1 JOIN aircraft AS T2 ...
pred: SELECT T2.Pilot_name, T1.Model FROM pilot_record AS T1 JOIN aircraft AS T2 ...
```

The query is fluent and plausible, but it does not execute. This is typical of a model that has learned the *surface form* of multi-table SQL from only about 4,000 Spider examples. Other recurring failures are ambiguous aggregation ("How many people led in points…" generated without COUNT; 4.8% of the test set), literal-span errors (*"Musician Panjabi Mc"* instead of *"Panjabi MC"*), and faithful reproduction of annotation noise: "smaller than 3,284" became *< 3 OFFSET 284* in the gold data, and the model reproduced it.

Variability. With nucleus sampling (Holtzman et al., 2020), five samples per question yield 1.28 distinct queries at temperature 0.7 and 1.67 at 1.3. Exact match falls from 83.0% to 77.1% over the same range. The learned distribution is sharply peaked, which is desirable here because a question usually has one correct query, and diversity mostly adds errors.

Ethical Considerations and Responsible Use

Misuse and security. A text-to-SQL generator turns natural language directly into database commands. If its output is executed with write privileges, a manipulated question (prompt injection) can produce destructive or data-leaking SQL. Pedro et al. (2023) demonstrate exactly such prompt-to-SQL injection attacks against applications integrated with large language models. Even without malice, our model generates fluent but wrong queries (about 11.5% of test cases). A wrong query looks just as authoritative as a correct one, so a non-expert user could act on silently incorrect numbers. Responsible deployment would require read-only database roles, an allow-list of statement types, showing the generated SQL to the user, and human review for consequential decisions.

Bias and societal impact. The model learns whatever patterns its training data contain (Bender et al., 2021). Our training data are 94% English Wikipedia sports, election and entertainment tables, and the model copied the annotation errors in them faithfully. Users who phrase questions in other dialects or languages, or who query other domains such as healthcare, will receive worse output, and the system gives no signal that it is outside its competence. Automating query writing can widen access to data for non-programmers, but it also shifts accountability for analytical errors onto people who cannot inspect the SQL. **Ownership:** the dataset is CC BY 4.0, and WikiSQL and Spider carry their own permissive licences,

so attribution, which this report provides, is required for any redistribution of the data.

Limitations and Future Improvements

The main limitations are the imbalance between simple and complex queries (only about 6% Spider), schemas that list only the needed columns, noisy gold labels, a small 60M-parameter model, and an execution metric based on one random database. Realistic next steps, in order of expected impact: (1) oversample Spider or add its full training set with complete database schemas, so the model must select columns among distractors; (2) clean the WikiSQL conversion artefacts (for example the OFFSET bug and bracketed column names) before training; (3) use constrained decoding that only allows tokens which keep the query grammatical and every identifier present in the schema, which would remove most invalid-SQL errors; (4) evaluate with distilled test suites (Zhong et al., 2020) for a tighter semantic metric; and (5) scale to CodeT5-base, or fine-tune a modern code LLM with parameter-efficient methods, given more GPU memory.

References

b-mc2. (2023). *sql-create-context* [Data set]. Hugging Face. https://huggingface.co/datasets/b-mc2/sql-create-context

Bender, E. M., Gebru, T., McMillan-Major, A., & Shmitchell, S. (2021). On the dangers of stochastic parrots: Can language models be too big? In *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency* (pp. 610–623). Association for Computing Machinery. https://doi.org/10.1145/3442188.3445922

Bowman, S. R., Vilnis, L., Vinyals, O., Dai, A. M., Jozefowicz, R., & Bengio, S. (2016). Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning* (pp. 10–21). Association for Computational Linguistics.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. In *Advances in Neural Information Processing Systems* (Vol. 27).

Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems* (Vol. 33, pp. 6840–6851).

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations*.

Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. In *International Conference on Learning Representations*.

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Pedro, R., Castro, D., Carreira, P., & Santos, N. (2023). *From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application?* arXiv. https://doi.org/10.48550/arXiv.2308.01990

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research, 21*(140), 1–67.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, ■., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30).

Wang, Y., Wang, W., Joty, S., & Hoi, S. C. H. (2021). CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing* (pp. 8696–8708). Association for Computational Linguistics.

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3911–3921). Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-1425

Zhong, R., Yu, T., & Klein, D. (2020). Semantic evaluation for text-to-SQL with distilled test suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing* (pp. 396–411). Association for Computational Linguistics.

Zhong, V., Xiong, C., & Socher, R. (2017). *Seq2SQL: Generating structured queries from natural language using reinforcement learning*. arXiv. https://doi.org/10.48550/arXiv.1709.00103